

IF2211 Teori Bahasa Formal dan Automata

Milestone 2 Syntax Analysis

Laporan Tugas Besar

Disusun untuk memenuhi tugas mata kuliah IF2224 Teori Bahasa Formal dan Otomata
pada Semester V Tahun Akademik 2025/2026



Disusun Oleh

Dita Maheswari	13523125
Boye Mangaratua Ginting	13523127
Samantha Laqueenna Ginting	13523138
Lukas Raja Agripa	13523158

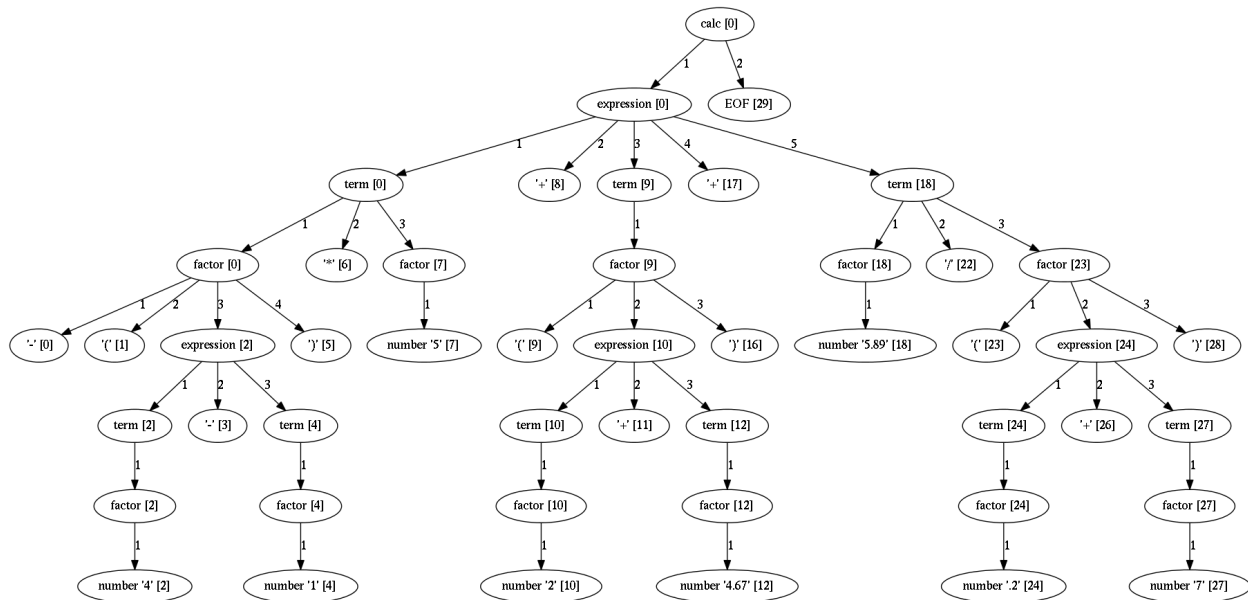
PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025

DAFTAR ISI

DAFTAR ISI.....	2
BAB 1 LANDASAN TEORI.....	3
BAB 2 PERANCANGAN DAN IMPLEMENTASI.....	5
a) Abstract Syntax Tree (AST).....	5
b) Declaration.....	5
c) Expression.....	10
d) Statement.....	13
e) Integrasi dengan Lexical Analyzer.....	14
f) Error Handling.....	15
BAB 3 PENGUJIAN.....	16
a) Input membaca file output .txt dari tokenisasi milestone 1.....	16
b) Input membaca program Pascal-S dari milestone 2.....	18
BAB 4 KESIMPULAN DAN SARAN.....	21
LAMPIRAN.....	22
A. Github.....	22
B. Pembagian Tugas.....	22
C. Grammar yang digunakan.....	22
REFERENSI.....	26

BAB 1 LANDASAN TEORI

Syntax Analysis (atau dikenal juga dengan istilah parsing) adalah tahap setelah Lexical Analysis. Dalam tahap sebelumnya, source code dipecah menjadi token-token singular oleh Lexer. Pada tahap ini, token-token tersebut akan dimaknai menjadi sebuah struktur tertentu. Syntax Analyzer atau Parser bertugas untuk memeriksa urutan token agar sesuai dengan aturan tata bahasa (grammar) dari bahasa pemrograman yang dikompilasi. Parser juga akan membangun sebuah struktur sintaks berbentuk pohon (Parse Tree) yang merepresentasikan struktur hierarkis source code berdasarkan grammar. Jika parser mendeteksi kesalahan sintaks, ia akan mengeluarkan pesan error (syntax error).



Contoh Parse Tree

Sumber: https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png

Berikut merupakan langkah-langkah di dalam tahap Syntax Analysis.

1. Parsing → Token dianalisis berdasarkan aturan tata bahasa bahasa pemrograman, dan parse tree atau AST dibangun yang mewakili struktur hierarki program. Salah satu metode algoritma yang umum digunakan untuk melakukan parsing ini adalah Recursive Descent Parsing. Ini adalah teknik parsing top-down di mana setiap bagian dari tata bahasa diimplementasikan sebagai satu prosedur atau fungsi. Fungsi-fungsi ini kemudian saling memanggil satu sama lain secara rekursif untuk memvalidasi urutan token masukan sesuai dengan grammar.
2. Error Handling → Jika program masukan mengandung kesalahan sintaksis, syntax analyzer mendeteksi dan melaporkannya kepada pengguna, disertai indikasi dimana kesalahan terjadi.

3. Symbol Table Creation → Syntax analyzer membuat tabel simbol, yang merupakan struktur data yang menyimpan informasi tentang pengenal yang digunakan dalam program, seperti jenis, cakupan, dan lokasinya.

BAB 2 PERANCANGAN DAN IMPLEMENTASI

a) Abstract Syntax Tree (AST)

AST merupakan representasi hierarkis dari struktur sintaksis program sumber yang sudah diproses oleh parser. AST digunakan untuk menyimpan hasil parsing dalam bentuk *tree* yang dimana:

- Setiap *node* mewakili satu konstruksi sintaks seperti <declaration-part>, <expression>, <if-statement>, dsb
- Hubungan antar *node* mencerminkan struktur *grammar* bahasa pemrograman yang sedang diproses

Dengan adanya *abstract syntax tree*, program selanjutnya dapat melakukan analisis semantik, optimasi kode, atau pembuatan *intermediate representation*. AST ini diimplementasikan pada *ast.py* yang berisi definisi kelas-kelas yang merepresentasikan struktur dari AST. Semua kelas ini diturunkan dari kelas dasar *ASTNode*, yang merupakan abstraksi umum dari setiap node pohon sintaks.

Kelas *ASTNode* berfungsi sebagai induk bagi seluruh node dalam AST. Setiap node di dalam pohon memiliki tipe node tertentu (*node_type*) dan turunannya (*children*). Kemudian terdapat metode *add_child()* yang digunakan setiap menambah turunan baik token maupun node lain ke node induknya.

b) Declaration

Class *DeclarationParser* merupakan sebuah kelas yang bertanggung jawab untuk melakukan parsing pada semua bagian tata bahasa yang terkait dengan deklarasi variabel, konstanta, tipe, dan subprogram (prosedur dan fungsi). Dengan memanfaatkan algoritma Recursive Descent, setiap method dalam kelas ini dirancang untuk melakukan parsing satu non-terminal spesifik dari grammar bahasa Pascal-S. Kelas ini diinisialisasi dengan sebuah referensi ke objek parser utama, agar kelas dapat memperoleh akses ke aliran token dan menggunakan token. Berikut merupakan penjelasan untuk masing-masing implementasi method deklarasi:

1. Struktur Hierarki Deklarasi

Parser ini menangani berbagai jenis deklarasi dalam urutan yang ditentukan oleh grammar Pascal-S.

- Deklarasi Konstanta
- Deklarasi Tipe
- Deklarasi Variabel
- Deklarasi Subprogram (Prosedur dan Fungsi)

2. Metode Parsing Utama

i. *parse_declaration_part()*

Metode ini melakukan parsing non-terminal <declaration-part>. Sesuai grammar, bagian deklarasi terdiri dari nol atau lebih deklarasi konstanta, tipe, variabel, dan subprogram yang muncul dalam urutan tersebut.

- Membuat *DeclarationPartNode*

- Memanggil `self.parse_const_declaration()` dalam loop while selama menemukan keyword “konstanta”
- Memanggil `self.parse_type_declaration()` dalam loop while selama menemukan keyword “tipe”
- Memanggil `self.parse_var_declaration()` dalam loop while selama menemukan keyword “variabel”
- Memanggil `self.parse_procedure_declaration()` atau `self.parse_function_declaration()` dalam loop while selama menemukan keyword “prosedur” atau “fungsi”

ii. **parse_const_declaration()**

Metode ini melakukan parsing non-terminal `<const-declaration>`. Grammar mendefinisikannya sebagai keyword konstanta diikuti oleh satu atau lebih pasangan identifier dan nilai yang dipisahkan semicolon.

- Membuat `ConstDeclarationNode`
- Mengharapkan dan menerima keyword “konstanta”
- Masuk ke loop untuk melakukan parsing setiap deklarasi konstanta:
 - Menerima `IDENTIFIER` (nama konstanta)
 - Menerima operator “=”
 - Menerima nilai konstanta (`NUMBER`, `CHAR_LITERAL`, `STRING_LITERAL`, `IDENTIFIER`, atau `KEYWORD`)
- Loop berlanjut selama token berikutnya adalah `IDENTIFIER`

iii. **parse_type_declaration()**

Metode ini melakukan parsing non-terminal `<type-declaration>`. Strukturnya mirip dengan `parse_const_declaration()`, namun untuk definisi tipe custom.

- Membuat `TypeDeclarationNode`
- Mengharapkan dan menerima keyword “tipe”
- Masuk ke loop untuk melakukan parsing setiap deklarasi tipe:
 - Menerima `IDENTIFIER` (nama tipe custom)
 - Menerima operator “=”
 - Memanggil `self.parse_type()` untuk melakukan parsing definisi tipe
 - Menerima `SEMICOLON`
- Loop berlanjut selama token berikutnya adalah `IDENTIFIER`

iv. **parse_var_declaration()**

Metode ini melakukan parsing non-terminal `<var-declaration>`. Metode ini menangani deklarasi variabel dengan format daftar identifier diikuti tipe data.

- Membuat `VarDeclarationNode`
- Mengharapkan dan mengonsumsi keyword “variabel”
- Masuk ke loop untuk mem-parsing setiap deklarasi variabel:
 - Memanggil `self.parse_identifier_list()` untuk mem-parsing daftar identifier yang dipisahkan koma

- Mengonsumsi COLON
 - Memanggil self.parse_type() untuk mem-parsing tipe data
 - Mengonsumsi SEMICOLON
 - Loop berlanjut selama token berikutnya adalah IDENTIFIER
- v. **parse_procedure_declaration()**
 Metode ini melakukan parsing non-terminal <procedure-declaration>. Prosedur adalah subprogram yang tidak mengembalikan nilai.
 - Membuat ProcedureDeclarationNode
 - Mengharapkan dan mengonsumsi keyword "prosedur"
 - Menerima IDENTIFIER (nama prosedur)
 - Jika token berikutnya adalah LPARENTHESIS, memanggil self.parse_formal_parameter_list() untuk mem-parsing parameter formal
 - Menerima SEMICOLON
 - Memanggil self.parse_declaration_part() secara rekursif untuk mem-parsing deklarasi lokal prosedur
 - Memanggil self.parser.statement_parser.parse_compound_statement() untuk mem-parsing body prosedur (mulai...selesai)
 - Menerima SEMICOLON penutup (jika ada)
- vi. **parse_function_declaration()**
 Metode ini melakukan parsing non-terminal <function-declaration>. Fungsi adalah subprogram yang mengembalikan nilai dengan tipe tertentu.
 - Membuat FunctionDeclarationNode
 - Mengharapkan dan mengonsumsi keyword "fungsi"
 - Mengonsumsi IDENTIFIER (nama fungsi)
 - Jika token berikutnya adalah LPARENTHESIS, memanggil self.parse_formal_parameter_list() untuk mem-parsing parameter formal
 - Menerima COLON
 - Memanggil self.parse_type() untuk mem-parsing tipe kembalian fungsi
 - Menerima SEMICOLON
 - Memanggil self.parse_declaration_part() secara rekursif untuk mem-parsing deklarasi lokal fungsi
 - Memanggil self.parser.statement_parser.parse_compound_statement() untuk mem-parsing body fungsi (mulai...selesai)
 - Menerima SEMICOLON penutup (jika ada)

3. Metode Parsing Tipe Data

i. parse_type()

Metode ini melakukan parsing non-terminal <type> dan menangani berbagai jenis tipe data yang didukung Pascal-S.

- Membuat TypeNode
 - Memeriksa jenis tipe:
 - Array: Jika keyword "larik", memanggil self.parse_array_type()
 - Record: Jika keyword "rekaman", memanggil self.parse_record_type()
 - Tipe Dasar: Jika keyword tipe primitif (integer, real, boolean, char, string, bulat, desimal, karakter, logika), token langsung ditambahkan
 - Subrange: Jika NUMBER atau IDENTIFIER diikuti RANGE_OPERATOR (..), memanggil self.parse_range()
 - Tipe Kustom: Jika hanya IDENTIFIER, token ditambahkan sebagai referensi tipe kustom
- ii. **parse_array_type()**
 Metode ini melakukan parsing non-terminal <array-type> untuk deklarasi array (larik).
- Membuat ArrayTypeNode
 - Mengharapkan dan mengonsumsi keyword "larik"
 - Menerima LBRACKET ([)
 - Memanggil self.parse_range() untuk melakukan parsing rentang indeks array
 - Menerima RBRACKET (])
 - Mengharapkan dan mengonsumsi keyword "dari"
 - Memanggil self.parse_type() untuk melakukan parsing tipe elemen array
- iii. **parse_range()**
 Metode ini melakukan parsing non-terminal <range> untuk rentang nilai subrange atau indeks array.
- Membuat RangeNode
 - Menerima nilai awal (NUMBER atau IDENTIFIER)
 - Mengharapkan dan mengonsumsi RANGE_OPERATOR (..)
 - Menerima nilai akhir (NUMBER atau IDENTIFIER)
- iv. **parse_record_type()**
 Metode ini melakukan parsing non-terminal <record-type> untuk deklarasi record (rekaman).
- Membuat RecordTypeNode
 - Mengharapkan dan mengonsumsi keyword "rekaman"
 - Memanggil self.parse_field_list() untuk melakukan parsing daftar field dalam record
 - Mengharapkan dan mengonsumsi keyword "selesai" sebagai penutup record
- v. **parse_field_list()**
 Metode ini melakukan parsing non-terminal <field-list> untuk daftar fields dalam record type.

- Membuat FieldListNode
- Memanggil self.parse_identifier_list() untuk mem-parsing daftar identifier field pertama
- Menerima COLON
- Memanggil self.parse_type() untuk mem-parsing tipe field
- Masuk ke loop while selama token saat ini adalah SEMICOLON:
 - Memeriksa apakah semicolon adalah trailing semicolon sebelum "selesai" (jika ya, tambahkan dan break)
 - Mengonsumsi SEMICOLON
 - Memanggil self.parse_identifier_list() untuk mem-parsing field berikutnya
 - Mengonsumsi COLON
 - Memanggil self.parse_type() untuk mem-parsing tipe field berikutnya

4. Metode Utilitas

i. parse_identifier_list()

Metode pembantu ini melakukan parsing non-terminal <identifier-list>, yaitu satu atau lebih identifier yang dipisahkan oleh koma.

- Membuat IdentifierListNode
- Mengharapkan dan mengonsumsi IDENTIFIER pertama
- Masuk ke loop while selama token saat ini adalah COMMA
- Di dalam loop, token COMMA ditambahkan, di-advance(), dan IDENTIFIER berikutnya dikonsumsi

ii. parse_formal_parameter_list()

Metode ini melakukan parsing non-terminal <formal-parameter-list> untuk parameter formal subprogram (prosedur/fungsi).

- Membuat FormalParameterListNode
- Mengharapkan dan mengonsumsi LPARENTHESIS
- Memeriksa apakah parameter list kosong (langsung RPARENTHESIS). Jika ya, tutup dan return
- Memeriksa keyword "variabel" untuk parameter pass-by-reference (VAR parameter). Jika ada, token ditambahkan
- Memanggil self.parse_identifier_list() untuk mem-parsing daftar parameter pertama
 - Menerima COLON
 - Memanggil self.parse_type() untuk mem-parsing tipe parameter
 - Masuk ke loop while selama token saat ini adalah SEMICOLON:
 - Menerima SEMICOLON
 - Memeriksa apakah masih ada parameter lain (jika RPARENTHESIS, break)
 - Memeriksa keyword "variabel" untuk parameter berikutnya (jika ada)

- Memanggil `self.parse_identifier_list()` untuk parameter group berikutnya
- Menerima COLON
- Memanggil `self.parse_type()` untuk tipe parameter berikutnya
- Mengharapkan dan menerima RPARENTHESIS

c) Expression

Class ExpressionParser merupakan sebuah kelas yang bertanggung jawab untuk melakukan parsing pada semua bagian tata bahasa yang terkait dengan ekspresi. Sesuai dengan method *Recursive Descent*, setiap *method* di dalam kelas ini dirancang untuk mem-*parsing* satu non-terminal spesifik dari tata bahasa Pascal-S. *Class ExpressionParser* diinisialisasi dengan sebuah referensi ke objek parser utama. Hal ini memberinya akses ke aliran token (seperti `self.parser.current_token`) dan metode menggunakan token (seperti `self.parser.advance( )`). Berikut penjelasan lebih lanjut mengenai implementasi expression :

1. Penanganan Operator Precedence

Struktur dari Recursive Descent Parser ini secara *inherent* menangani operator precedence (prioritas operator). *Grammar* untuk ekspresi dirancang dalam beberapa tingkatan:

- i. `<expression>`
Menangani operator dengan prioritas terendah seperti =, >, <
- ii. `<simple-expression>`
Menangani operator +, -, dan 'atau'
- iii. `<term>`
Menangani operator *, /, bagi, mod, dan 'dan'
- iv. `<factor>`
Menangani unit dasar seperti angka, variabel, literal, ekspresi dalam kurung, dan pemanggilan fungsi

Struktur pemanggilan fungsinya adalah `parse_expression()` memanggil `parse_simple_expression()`, yang memanggil `parse_term()`, yang memanggil `parse_factor()`. Hal ini memastikan bahwa operasi dengan prioritas lebih tinggi (misalnya, perkalian di `parse_term`) dievaluasi terlebih dahulu sebelum operasi dengan prioritas lebih rendah (misalnya, penjumlahan di `parse_simple_expression`).

2. Metode Parsing Utama

- i. **`parse_expression()`**
Metode ini mem-*parsing* non-terminal `<expression>`. Sesuai tata bahasa, sebuah ekspresi adalah satu `<simple-expression>` yang opsional diikuti oleh `<relational-operator>` dan `<simple-expression>` kedua.
 - Membuat *ExpressionNode*
 - Memanggil `self.parse_simple_expression()` untuk mem-*parsing* ekspresi sederhana pertama dan menambahkannya sebagai child

- Memeriksa apakah token saat ini adalah `RELATIONAL_OPERATOR`
- Jika ya, token operator tersebut ditambahkan sebagai child, parser di-*advance()*, dan *self.parse_simple_expression()* dipanggil lagi untuk mem-parsing ekspresi sederhana kedua

ii. ***parse_simple_expression()***

Metode ini mem-parsing non-terminal `<simple-expression>`. Tata bahasa mendefinisikannya sebagai `<term>` yang diawali oleh operator unary (+ atau -) opsional, dan diikuti oleh nol atau lebih `<additive-operator>` dan `<term>`.

- Membuat *SimpleExpressionNode*
- Memeriksa unary + atau - di awal. Jika ada, token tersebut ditambahkan dan di-*advance()*
- Memanggil *self.parse_term()* untuk mem-parsing `<term>` pertama
- Masuk ke loop while selama token saat ini adalah operator aditif (dicek menggunakan *is_additive_operator()*)
- Di dalam loop, token operator ditambahkan, di-*advance()*, dan *self.parse_term()* dipanggil untuk mem-parsing `<term>` berikutnya

iii. ***parse_term()***

Metode ini mem-parsing non-terminal `<term>`. Strukturnya mirip dengan *parse_simple_expression()*, namun untuk operator multiplikatif.

- Membuat *TermNode*
- Memanggil *self.parse_factor()* untuk mem-parsing `<factor>` pertama
- Masuk ke loop while selama token saat ini adalah operator multiplikatif (dicek menggunakan *is_multiplicative_operator()*)
- Di dalam loop, token operator ditambahkan, di-*advance()*, dan *self.parse_factor()* dipanggil untuk mem-parsing `<factor>` berikutnya

iv. ***parse_factor()***

Metode ini adalah unit dasar dari rekursi ekspresi. Metode ini mem-parsing non-terminal `<factor>` dan menangani berbagai kemungkinan atomik:

- Literals: Jika token adalah `NUMBER`, `CHAR_LITERAL`, atau `STRING_LITERAL`, token tersebut langsung ditambahkan sebagai child dari *FactorNode*.
- Operator "tidak": Jika token adalah `LOGICAL_OPERATOR` dengan nilai "tidak", token ditambahkan, lalu *self.parse_factor()* dipanggil secara rekursif untuk mem-parsing faktor yang dinegasikan.

- Ekspresi Berkurung: Jika token adalah LPARENTHESIS (`()`), token tersebut ditambahkan, *self.parse_expression()* dipanggil secara rekursif untuk mem-parsing sub-ekspresi di dalamnya. Terakhir, parser akan mengharapkan dan mengonsumsi RPARENTHESIS (`()`).
- Identifier (Variabel, Panggilan Fungsi, Array, Record): Ini adalah bagian paling kompleks dari *parse_factor*.
 - Jika token adalah IDENTIFIER (atau KEYWORD untuk boolean `true/false`), implementasi melakukan lookahead (mengintip) satu token ke depan (*next_pos*).
 - Jika token berikutnya adalah LPARENTHESIS (`()`), itu adalah panggilan fungsi. Parser kemudian memanggil *self.parser.statement_parser.parse_call_statement(is_function=True)* untuk mem-parsing seluruh struktur panggilan fungsi.
 - Jika token berikutnya adalah LBRACKET (`[]`), itu adalah akses array. Parser mengonsumsi IDENTIFIER, `[`, memanggil *self.parse_expression()* untuk mem-parsing ekspresi indeks, lalu mengonsumsi `]`.
 - Jika token berikutnya adalah DOT (`.`), itu adalah akses field record. Parser mengonsumsi IDENTIFIER pertama, lalu masuk ke loop untuk mengonsumsi setiap pasang `.` dan IDENTIFIER berikutnya (untuk menangani nested record seperti `record.field.subfield`).
 - Jika tidak satu pun di atas, itu adalah variabel biasa (atau boolean literal). Token IDENTIFIER ditambahkan.

3. Metode Utilitas

i. *is_additive_operator(token)* dan *is_multiplicative_operator(token)*

Dua metode pembantu ini digunakan untuk memeriksa apakah sebuah token termasuk dalam kategori operator aditif (+, -, atau) atau multiplikatif (*, /, bagi, mod, dan). Ini menyederhanakan logika loop while di dalam *parse_simple_expression()* dan *parse_term()*.

ii. *parse_parameter_list()*

Metode ini dipanggil oleh parser pemanggilan fungsi/prosedur. Metode ini mem-parsing non-terminal `<parameter-list>`, yang didefinisikan sebagai satu atau lebih `<expression>` yang dipisahkan oleh COMMA.

- Membuat *ParameterListNode*
- Memanggil *self.parse_expression()* untuk mem-parsing parameter pertama
- Masuk ke loop while selama token saat ini adalah COMMA

- Di dalam loop, token COMMA ditambahkan, di-*advance()*, dan *self.parse_expression()* dipanggil untuk mem-parsing ekspresi parameter berikutnya.

d) Statement

Bagian *Statement* bertanggung jawab untuk melakukan *parsing* terhadap konstruksi kode yang dapat dieksekusi (*executable code*) di dalam program. Ini mencakup *assignment*, pemanggilan prosedur, struktur kontrol (seperti *if*, *while*, *for*), dan blok program (mulai...selesai). Implementasi logika ini terdapat dalam file *src/Parser/statements.py* pada kelas *StatementParser*.

Kelas *StatementParser* diinisialisasi dengan *instance* dari *parser* utama, yang memberinya akses ke *stream token*, *helper method* (*advance()*), dan *sub-parser* lain seperti *ExpressionParser*.

Metode inti dari kelas ini adalah *parse_statement()*, yang berfungsi sebagai *dispatcher*. Dengan menggunakan pendekatan *Recursive Descent*, metode ini menganalisis token saat ini (*current_token*) untuk menentukan jenis *statement* yang akan diparsing:

1. **KEYWORD Dispatch:** Jika token saat ini adalah KEYWORD, *dispatcher* akan memanggil fungsi spesifik berdasarkan nilai *keyword* tersebut.
 - 'mulai' memanggil *parse_compound_statement()*.
 - 'jika' memanggil *parse_if_statement()*.
 - 'selama' memanggil *parse_while_statement()*.
 - 'untuk' memanggil *parse_for_statement()*.
 - 'writeln' (sebagai *built-in*) memanggil *parse_call_statement()*.
2. **IDENTIFIER Dispatch:** Tantangan utama terjadi saat token saat ini adalah IDENTIFIER, karena ini bisa menandakan dua konstruksi yang berbeda: *assignment-statement* (misal, *x := 5*) atau *procedure-call* (misal, *cek_hasil(x)*).

Untuk mengatasi ambiguitas ini, sebuah *helper method* privat, *_peek_token()*, diimplementasikan. Metode ini melakukan *lookahead* satu token ke depan tanpa memajukan *pointer* token (*stream*).

- Jika token berikutnya adalah ASSIGN_OPERATOR (*:=*), *dispatcher* akan memanggil *parse_assignment_statement()*.
- Jika token berikutnya adalah LPARENTHESIS (*()*), *dispatcher* akan memanggil *parse_call_statement()*. Keputusan ini sejalan dengan spesifikasi Milestone 2 yang mewajibkan penggunaan tanda kurung pada pemanggilan fungsi/prosedur.

Setiap fungsi *parsing* spesifik (contoh: *parse_if_statement()*) dirancang untuk mengimplementasikan tata bahasa (grammar) yang relevan. Sebagai contoh, *parse_if_statement* akan:

1. Memakai *token* KEYWORD(jika).
2. Memanggil *self.parser.expression_parser.parse_expression()* untuk mem-parsing kondisi.
3. Memakai *token* KEYWORD(maka).

4. Memanggil *self.parse_statement()* secara rekursif untuk mem-parsing *statement* di dalam blok *if*.
5. Secara opsional, memeriksa keberadaan *token* KEYWORD(selain-itu) dan memanggil *self.parse_statement()* sekali lagi jika ada.

Blok *compound* (mulai...selesai) ditangani oleh *parse_compound_statement()* yang memanggil *parse_statement_list()* untuk mengurai serangkaian *statement* yang dipisahkan oleh SEMICOLON. Setiap fungsi ini akan membuat *node* AST yang sesuai (misal: *IfStatementNode*, *AssignmentStatementNode*, *CompoundStatementNode*) dan menambahkannya ke pohon sintaksis untuk digunakan pada tahap analisis semantik selanjutnya.

e) Integrasi dengan Lexical Analyzer

Lexical Analyzer yang telah diimplementasikan sebelumnya, meninjau dari bagian Lexer.py dan Token.py yang masing masing bertanggung jawab untuk membaca source code (.pas) per karakter, mengelompokkan karakter menjadi token yang bermakna, mengidentifikasi jenis token dan menyimpan informasi posisi token serta representasi dari stauan leksikal terkecil dalam source code. Kemudian melalui class Syntax Parser yang terletak pada Parser/parser.py yang merupakan komponen menerima list of token (Token[]) dari lexer, kemudian diproses token secara sekuensial menggunakan Recursive Descent, membangun AST / Parse Tree, dan melakukan validasi sintaks sesuai grammar yang didefinisikan. Kalau mulai dari awal, dari source code .pas, diinput melalui entry point (main.py) akan di return sebagai string, kemudian akan melakukan LoadDFA dari JSON dan menginisialisasi Lexer dengan DFA sebagaimana telah dienkup seperti yang telah saya sampaikan sebelumnya melalui laporan lalu. Dan akan dijalankan method performLexicalAnalysis() untuk setSourceCode(sourceCode), dan mendapatkan Token dari lexer.tokenize() dan akhirnya di return List of Token (Token[]),

List of Token (Token[]) yang telah berisi token dari source code, akan dilakukan syntax analysis, dari class SyntaxAnalyzer yang deconstruct berisikan object Token, poisisi, current_token inisial Token[0], Inisialisasi sub-parser, setelah dibentuk object SyntaxAnalyzer, akan dipanggil method parse() untuk memproses token satu persatu dan build parse tree nodes. Kemudian, dilakukan Parse Tree (AST), ProgramNode dengan child node. Apabila ingin mendapatkan secara rinci program dari Parser sebagai SyntaxAnalyzer, dapat anda cek dipoin sebelumnya, untuk Token dan Lexer di bagian laporan sebelumnya.

Singkatnya, method key integrasi disini adalah berikut :

```
def initializeLexer(self, dfaFilePath: str):
    if not self.dfa:
        self.loadDFA(dfaFilePath)
    self.lexer = Lexer(self.dfa)

def performLexicalAnalysis(self, sourceCode: str) -> List[Token]:
    self.lexer.setSourceCode(sourceCode)
```

```
tokens = self.lexer.tokenize()
return tokens
```

```
def performSyntaxAnalysis(self, tokens: List[Token]) -> Optional[ProgramNode]:
    syntaxParser = SyntaxParser(tokens)
    parseTree = syntaxParser.parse()
    return parseTree
```

f) Error Handling

Modul ini mendefinisikan tiga kelas pengecualian utama, yang semuanya mewarisi dari kelas Exception bawaan Python, yaitu:

1. **Class *ParseError* :**

ParseError adalah kelas dasar untuk semua kesalahan yang terkait dengan parsing, yaitu menyediakan fungsionalitas inti yang sama untuk semua jenis kesalahan sintaksis, yaitu menyimpan pesan kesalahan, nomor baris, dan nomor kolom.

2. **Class *UnexpectedTokenError* :**

UnexpectedTokenError adalah pengecualian yang paling umum dan penting dalam parser. Kelas ini mewarisi dari *ParseError*, yaitu melempar *exception* ketika parser mengharapkan satu atau beberapa jenis token tertentu, tetapi malah menemukan token yang berbeda.

3. **Class *UnexpectedEOFError* :**

UnexpectedEOFError adalah kasus khusus dan turunan dari *ParseError* yang dilemparkan ketika parser mencapai akhir file (atau akhir aliran token) secara tak terduga. Kelas ini memberi sinyal error ketika parser masih membutuhkan lebih banyak token untuk menyelesaikan sebuah aturan tata bahasa (misalnya, program selesai tanpa selesai.), tetapi tidak ada lagi token yang tersisa.

BAB 3 PENGUJIAN

a) Input membaca file output .txt dari tokenisasi milestone 1

1. Program1.txt

Input	<pre>test > milestone-1 > output > Program1.txt You, 53 minutes ago 2 authors (riukassa and one other) 1 KEYWORD(program) 2 IDENTIFIER(helloworld) 3 SEMICOLON(;) 4 KEYWORD(variabel) 5 IDENTIFIER(a) 6 COMMA(,) 7 IDENTIFIER(b) 8 COLON(:) 9 KEYWORD(integer) 10 SEMICOLON(;) 11 KEYWORD(mulai) 12 IDENTIFIER(a) 13 ASSIGN_OPERATOR(=) 14 NUMBER(-0.123E) 15 SEMICOLON(;) 16 IDENTIFIER(b) 17 ASSIGN_OPERATOR(=) 18 IDENTIFIER(a) 19 ARITHMETIC_OPERATOR(+) 20 NUMBER(10) 21 SEMICOLON(;) 22 IDENTIFIER(writeln) 23 LPARENTHESIS((24 STRING_LITERAL('Result = ') 25 COMMA(,) 26 IDENTIFIER(b) 27 RPARENTHESIS()) 28 SEMICOLON(;) 29 KEYWORD(selesai) 30 DOT(.)</pre>
Output	(output lebih lengkap sudah ada di test/milestone-2/output/Program1.txt)

	<pre> VICTUS 16@LAPTOP-A8M3C9QR MINGW64 ~/OneDrive/Documents/PSC-Tubes-IF2224/src (feat/mils2) \$ python run.py ../test/milestone-1/output/Program1.txt Mode: Reading tokens from file: ../test/milestone-1/output/Program1.txt 30 tokens berhasil di-load dari ../test/milestone-1/output/Program1.txt === Parse Tree === <program> ├── <program-header> │ ├── KEYWORD(program) │ ├── IDENTIFIER(helloworld) │ └── SEMICOLON(;) ├── <declaration-part> │ ├── <var-declaration> │ │ ├── KEYWORD(variabel) │ │ ├── <identifier-list> │ │ │ ├── IDENTIFIER(a) │ │ │ ├── COMMA(,) │ │ │ └── IDENTIFIER(b) │ │ ├── COLON(:) │ │ ├── <type> │ │ │ └── KEYWORD(integer) │ │ └── SEMICOLON(;) │ └── <compound-statement> │ ├── KEYWORD(mulai) │ └── <statement-list> │ ├── <assignment-statement> │ │ ├── IDENTIFIER(a) │ │ ├── ASSIGN_OPERATOR(:=) │ │ └── <expression> │ │ ├── <simple-expression> │ │ │ ├── <term> │ │ │ └── <factor> │ │ │ └── NUMBER(-0.123E) │ │ └── SEMICOLON(;) │ ├── <assignment-statement> │ │ ├── IDENTIFIER(b) │ │ ├── ASSIGN_OPERATOR(:=) │ │ └── <expression> │ │ ├── <simple-expression> │ │ │ ├── <term> │ │ │ └── <factor> </pre>
--	---

2. Program5.txt

Input	<pre> test > milestone-1 > output > Program5.txt You, 54 minutes ago 2 authors (rlukassa and one other) 1 KEYWORD(program) 2 IDENTIFIER(bebas) 3 SEMICOLON(;) 4 KEYWORD(variabel) 5 IDENTIFIER(ch) 6 COLON(:) 7 KEYWORD(char) 8 SEMICOLON(;) 9 KEYWORD(mulai) 10 IDENTIFIER(ch) 11 ASSIGN_OPERATOR(:=) 12 CHAR_LITERAL('A') 13 SEMICOLON(;) 14 IDENTIFIER(writeIn) 15 LPARENTHESIS((16 STRING_LITERAL('Dont panic: value = ') 17 COMMA(,) 18 IDENTIFIER(ch) 19 RPARENTHESIS()) 20 SEMICOLON(;) 21 KEYWORD(selesai) 22 DOT(.) </pre>
Output	(output lebih lengkap sudah ada di test/milestone-2/output/Program5.txt)

	<pre> VICTUS 16@LAPTOP-ABM3C9QR MINGW64 ~/OneDrive/Documents/PSC-Tubes-IF2224/src (feat/mls2) • \$ python run.py ../test/milestone-1/output/Program5.txt Mode: Reading tokens from file: ../test/milestone-1/output/Program5.txt 22 tokens berhasil di-load dari ../test/milestone-1/output/Program5.txt === Parse Tree === <program> ├── <program-header> │ ├── KEYWORD(program) │ ├── IDENTIFIER(bebas) │ └── SEMICOLON(;) ├── <declaration-part> │ ├── <var-declaration> │ │ ├── KEYWORD(variabel) │ │ ├── <identifier-list> │ │ │ └── IDENTIFIER(ch) │ │ ├── COLON(:) │ │ ├── <type> │ │ │ └── KEYWORD(char) │ │ └── SEMICOLON(;) │ └── <compound-statement> │ ├── KEYWORD(mulai) │ ├── <statement-list> │ │ ├── <assignment-statement> │ │ │ ├── IDENTIFIER(ch) │ │ │ ├── ASSIGN_OPERATOR(:=) │ │ │ └── <expression> │ │ │ ├── <simple-expression> │ │ │ │ ├── <term> │ │ │ │ │ └── <factor> │ │ │ │ │ └── CHAR_LITERAL('A') │ │ │ └── SEMICOLON(;) │ │ └── <procedure-call> │ │ ├── IDENTIFIER(writeln) │ │ ├── LPARENTHESIS(() │ │ └── <parameter-list> │ │ └── <expression> </pre>
--	---

b) Input membaca program Pascal-S dari milestone 2

1. Program8.pas

Input	<pre> test > milestone-2 > input > Program8.pas You, 2 days ago 1 author (You) 1 program TestIf; 2 variabel 3 x, y: integer; 4 mulai 5 x := 10; 6 jika x > 5 maka 7 y := 1 8 selain-itu 9 y := 0; 10 writeln('Y = ', y); 11 selesai. </pre>
Output	(output lebih lengkap sudah ada di test/milestone-2/output/Program8.txt)

	<pre> VICTUS 16@LAPTOP-A8M3C9QR MINGW64 ~/OneDrive/Documents/PSC-Tubes-IF2224/src (feat/mils2) • \$ python run.py ../test/milestone-2/input/Program8.pas Mode: Processing Pascal file: ../test/milestone-2/input/Program8.pas === Parse Tree === <program> ├── <program-header> │ ├── KEYWORD(program) │ ├── IDENTIFIER(TestIf) │ └── SEMICOLON(;) ├── <declaration-part> │ ├── <var-declaration> │ │ ├── KEYWORD(variabel) │ │ ├── <identifier-list> │ │ │ ├── IDENTIFIER(x) │ │ │ ├── COMMA(,) │ │ │ └── IDENTIFIER(y) │ │ ├── COLON(:) │ │ ├── <type> │ │ │ └── KEYWORD(integer) │ │ └── SEMICOLON(;) │ └── <compound-statement> │ ├── KEYWORD(mulai) │ └── <statement-list> │ ├── <assignment-statement> │ │ ├── IDENTIFIER(x) │ │ ├── ASSIGN_OPERATOR(:=) │ │ └── <expression> │ │ ├── <simple-expression> │ │ │ └── <term> │ │ │ └── <factor> │ │ │ └── NUMBER(10) │ │ └── SEMICOLON(;) │ ├── <if-statement> │ │ ├── KEYWORD(jika) │ │ └── <expression> │ │ ├── <simple-expression> │ │ │ ├── <term> │ │ │ │ ├── <factor> │ │ │ │ └── IDENTIFIER(x) │ │ └── RELATIONAL_OPERATOR(>) │ └── <simple-expression> </pre>
--	---

2. Program9.pas

Input	<pre> test > milestone-2 > input > Program9.pas You, 4 minutes ago 1 author (You) 1 program TestArray; 2 variabel 3 numbers: larik[1..10] dari integer; 4 i: integer; 5 mulai 6 untuk i := 1 ke 10 lakukan 7 numbers := i * 2; 8 writeln('Done'); 9 selesai. </pre>
Output	(output lebih lengkap sudah ada di test/milestone-2/output/Program9.txt)

	<pre> VICTUS 16@LAPTOP-A8M3C9QR MINGW64 ~/OneDrive/Documents/PSC-Tubes-IF2224/src (feat/mils2) \$ python run.py ../test/milestone-2/input/Program9.pas === Parse Tree === <program> ├── <program-header> │ ├── KEYWORD(program) │ ├── IDENTIFIER(TestArray) │ └── SEMICOLON(;) ├── <declaration-part> │ ├── <var-declaration> │ │ ├── KEYWORD(variabel) │ │ ├── <identifier-list> │ │ │ └── IDENTIFIER(numbers) │ │ ├── COLON(:) │ │ └── <type> │ │ ├── <array-type> │ │ │ ├── KEYWORD(larik) │ │ │ ├── LBRACKET([) │ │ │ ├── <range> │ │ │ │ ├── NUMBER(1) │ │ │ │ ├── RANGE_OPERATOR(..) │ │ │ │ └── NUMBER(10) │ │ │ └── RBRACKET(]) │ │ ├── KEYWORD(dari) │ │ └── <type> │ │ └── KEYWORD(integer) │ └── SEMICOLON(;) ├── <identifier-list> │ └── IDENTIFIER(i) ├── COLON(:) ├── <type> │ └── KEYWORD(integer) └── SEMICOLON(;) <compound-statement> ├── KEYWORD(mulai) ├── <statement-list> │ └── <for-statement> │ ├── KEYWORD(untuk) │ ├── IDENTIFIER(i) │ ├── ASSIGN_OPERATOR(:=) │ ├── <expression> │ └── <simple-expression> </pre>
--	--

3. Program10.pas (akan error karena tidak sesuai aturan *grammar*)

Input	<pre> test > milestone-2 > input > ≡ Program10.pas You, 5 hours ago 1 author (You) 1 program ErrorTest; 2 variabel 3 x: integer 4 mulai 5 x := 5; 6 selesai. </pre>
Output	<pre> VICTUS 16@LAPTOP-A8M3C9QR MINGW64 ~/OneDrive/Documents/PSC-Tubes-IF2224/src (feat/mils2) \$ python run.py ../test/milestone-2/input/Program10.pas Mode: Processing Pascal file: ../test/milestone-2/input/Program10.pas ERROR: Syntax error: Syntax error at line 4, column 1: unexpected token KEYWORD(mulai), expected SEMICOLON Gagal </pre> Syntax error karena tidak ada SEMICOLON (;) setelah 'x : integer'

BAB 4 KESIMPULAN DAN SARAN

a) Kesimpulan

Pengerjaan Milestone 2 ini telah berhasil mengimplementasikan tahap *Syntax Analysis* (Analisis Sintaksis) untuk *compiler* PASCAL-S. Proses analisis ini dikerjakan dengan menggunakan metode *Recursive Descent* untuk mem-parsing *list of tokens* yang merupakan keluaran dari *lexer* pada Milestone 1.

Fungsi utama *parser* yang dibuat adalah untuk memverifikasi apakah urutan *token-token* tersebut sesuai dengan tata bahasa (grammar) PASCAL-S yang telah didefinisikan. Ketika urutan *token* valid, *parser* akan berhasil membangun sebuah *Parse Tree* (Pohon Sintaksis) yang merepresentasikan struktur hierarkis dari program sumber. Pohon ini menjadi landasan penting untuk tahap kompilasi berikutnya.

Implementasi *parser* dibagi secara modular (*parser* deklarasi, *statement*, dan *expression*) untuk mengelola kompleksitas tata bahasa dan memfasilitasi pengerjaan tim. Selain itu, *parser* juga telah dilengkapi dengan mekanisme deteksi *error* yang akan memberikan pesan informatif apabila menemukan urutan *token* yang tidak valid, sehingga membantu *programmer* mengidentifikasi kesalahan sintaksis.

b) Saran

Adapun saran yang dapat dipertimbangkan untuk pengembangan di lain waktu ialah:

1. **Peningkatan *Error Handling*:** Meskipun deteksi *error* dasar (melaporkan *error* pertama dan berhenti) telah diimplementasikan, *parser* dapat ditingkatkan dengan mekanisme *error recovery*. Kemampuan ini akan memungkinkan *parser* untuk melanjutkan proses *parsing* setelah menemukan kesalahan, sehingga dapat melaporkan beberapa kesalahan sintaksis sekaligus dalam satu kali eksekusi.
2. **Abstraksi *Node AST*:** *Parse Tree* yang dihasilkan saat ini masih sangat konkret dan erat kaitannya dengan tata bahasa. Untuk mempermudah tahap *Semantic Analysis*, *Parse Tree* ini dapat diabstraksi lebih lanjut menjadi *Abstract Syntax Tree* (AST) yang lebih ringkas dan fokus pada makna semantik, bukan hanya sintaksis murni.
3. **Penguatan Kontrak Antar Modul:** Pengerjaan modular (*deklarasi*, *statement*, *expression*) terbukti efektif. Namun, pendekatan ini sangat bergantung pada "kontrak" atau antarmuka (API) antar modul (misalnya, `parse_expression()` yang dipanggil oleh `parse_statement()`). Disarankan untuk selalu mematangkan definisi antarmuka ini di awal agar proses integrasi berjalan lebih lancar.

LAMPIRAN

A. Github

<https://github.com/DitaMaheswari05/PSC-Tubes-IF2224.git>

B. Pembagian Tugas

Nama / NIM	AST dan ErrorHandling	Parser	Integrasi dengan Lexer	Laporan
Dita Maheswari / 13523125	100%	20%	-	25%
Boye Mangaratua Ginting / 13523127	-	30%	-	25%
Samantha Laqueenna Ginting / 13523138	-	30%	-	25%
Lukas Raja Agripa / 13523158	-	20%	100%	25%
Total	100%	100%	100%	100%

NIM	Nama	Pekerjaan
13523125	Dita Maheswari	Expression parser, AST, ErrorHandling, laporan
13523127	Boye Mangaratua Ginting	Statement parser, laporan
13523138	Samantha Laqueenna Ginting	Declaration parser, laporan
13523158	Lukas Raja Agripa	Main parser, Integrasi Lexical Analyzer, laporan

C. Grammar yang digunakan

Kami menggunakan $:=$ untuk definisi, $+$ untuk rangkaian (sequence), $|$ untuk pilihan (alternation), $*$ untuk 0 atau lebih, $^+$ untuk 1 atau lebih, dan $?$ untuk opsional.

Berikut adalah daftar lengkap *grammar rules* yang kami gunakan untuk mem-parsing bahasa Pascal-S

1. Struktur Program Utama

<code><program> ::= <program-header> <declaration-part> <compound-statement> DOT</code>
<code><program-header> ::= KEYWORD(program) IDENTIFIER SEMICOLON</code>

2. Bagian Deklarasi

<code><declaration-part> ::= (<const-declaration>)* (<type-declaration>)* (<var-declaration>)* (<subprogram-declaration>)*</code>
<code><subprogram-declaration> ::= <procedure-declaration> <function-declaration></code>

a. Deklarasi Konstanta

<code><const-declaration> ::= KEYWORD(konstanta) (<const-assignment>)+</code>
<code><const-assignment> ::= IDENTIFIER RELATIONAL_OPERATOR(=) <constant> SEMICOLON</code>
<code><constant> ::= NUMBER CHAR_LITERAL STRING_LITERAL IDENTIFIER KEYWORD(true) KEYWORD(false)</code>

b. Deklarasi Tipe

<code><type-declaration> ::= KEYWORD(tipe) (<type-definition>)+</code>
<code><type-definition> ::= IDENTIFIER RELATIONAL_OPERATOR(=) <type> SEMICOLON</code>
<code><type> ::= <simple-type> <array-type> <record-type> <range></code>
<code><simple-type> ::= KEYWORD(integer) KEYWORD(real) KEYWORD(boolean) KEYWORD(char) KEYWORD(string) KEYWORD(bulat) KEYWORD(desimal) KEYWORD(karakter) KEYWORD(logika) IDENTIFIER</code>
<code><array-type> ::= KEYWORD(larik) LBRACKET <range> RBRACKET KEYWORD(dari) <type></code>
<code><range> ::= <constant> RANGE_OPERATOR(..) <constant></code>
<code><record-type> ::= KEYWORD(rekaman) <field-list> KEYWORD(selesai)</code>
<code><field-list> ::= <field> (SEMICOLON <field>)* (SEMICOLON)?</code>
<code><field> ::= <identifier-list> COLON <type></code>

c. Statement percabangan dan perulangan

<code><if-statement> ::= KEYWORD(jika) <expression> KEYWORD(maka) <statement> (KEYWORD(selain-itu) <statement>)?</code>
<code><while-statement> ::= KEYWORD(selama) <expression> KEYWORD(lakukan) <statement></code>
<code><for-statement> ::= KEYWORD(untuk) IDENTIFIER ASSIGN_OPERATOR(=) <expression> <for-direction> <expression> KEYWORD(lakukan) <statement></code>
<code><for-direction> ::= KEYWORD(ke) KEYWORD(turun-ke)</code>
<code><repeat-statement> ::= KEYWORD(ulangi) <statement-list> KEYWORD(sampai) <expression></code>
<code><case-statement> ::= KEYWORD(kasus) <expression> KEYWORD(dari) <case-element> (SEMICOLON <case-element>)* (SEMICOLON)? KEYWORD(selesai)</code>
<code><case-element> ::= <case-label-list> COLON <statement></code>
<code><case-label-list> ::= <constant> (COMMA <constant>)*</code>

4. Bagian Ekspresi

<code><expression> ::= <simple-expression> (<relational-operator> <simple-expression>)?</code>
<code><simple-expression> ::= (ARITHMETIC_OPERATOR(+) ARITHMETIC_OPERATOR(-))? <term> (<additive-operator> <term>)*</code>
<code><term> ::= <factor> (<multiplicative-operator> <factor>)*</code>
<code><factor> ::= <variable> <function-call> NUMBER CHAR_LITERAL STRING_LITERAL KEYWORD(true) KEYWORD(false) LPARENTHESIS <expression> RPARENTHESIS LOGICAL_OPERATOR(tidak) <factor></code>

a. Variabel, Pemanggilan Fungsi, dan Operator

<code><variable> ::= IDENTIFIER (<field-access> <array-access>)*</code>
<code><field-access> ::= DOT IDENTIFIER</code>
<code><array-access> ::= LBRACKET <expression> RBRACKET</code>
<code><function-call> ::= IDENTIFIER LPARENTHESIS (<parameter-list>)? RPARENTHESIS</code>

<pre><relational-operator> ::= RELATIONAL_OPERATOR(=) RELATIONAL_OPERATOR(<)> RELATIONAL_OPERATOR(<) RELATIONAL_OPERATOR(<=) RELATIONAL_OPERATOR(>) RELATIONAL_OPERATOR(>=)</pre>

<pre><additive-operator> ::= ARITHMETIC_OPERATOR(+) ARITHMETIC_OPERATOR(-) LOGICAL_OPERATOR(atau)</pre>

<pre><multiplicative-operator> ::= ARITHMETIC_OPERATOR(*) ARITHMETIC_OPERATOR(/) ARITHMETIC_OPERATOR(bagi) ARITHMETIC_OPERATOR(mod) LOGICAL_OPERATOR(dan)</pre>

REFERENSI

- Introduction to Syntax Analysis in Compiler Design
<https://www.geeksforgeeks.org/compiler-design/introduction-to-syntax-analysis-in-compiler-design/>
- Recursive-Descent Parsing
https://www.cs.rochester.edu/u/nelson/courses/csc_173/grammars/parsing.html
- Youtube: Introduction Syntax Analyzer, Neso Academy
https://www.youtube.com/watch?v=3rzTRtjUM_I&list=PLBInK6fEyqRhMjOLYfqGdyB7Gt_k5cD6t